Overview of Prompt Development
After creating the first draft of a prompt (Version 1), it must undergo rigorous testing, tuning, and proper storing. A prompt is never ready for deployment after just a single successful input and output; it requires a systematic evaluation process.

Creating a Robust Dataset

- **Versatility is Key:** Testing requires a carefully curated, versatile dataset that covers all possible input types. For a review-analysis LLM, this means including positive, negative, and neutral reviews across multiple different departments.
- **Edge Cases:** The dataset must include confusing inputs and edge cases, such as reviews containing sarcasm. For example, a review stating, "I want to personally thank whoever designed this shirt for making me look like a stuffed sausage," uses positive words but holds a distinctly negative sentiment.
- **Cross-Team Collaboration:** A comprehensive dataset is rarely built by one person; it is usually developed with input from product, legal, market, and research teams.

Testing Methods Testing evaluates how well the LLM performs by comparing the "expected output" with the "actual output" generated by the model. There are two primary ways to test:

1. **Human Testing:** A human manually reads the LLM's output to verify if it aligns with the exact needs of the user, such as checking if the output is in the correct JSON format and contains the exact number of required keys.
2. **LLM-as-a-judge:** Using another AI model to evaluate the output.

Structuring Prompts and Security Protocols

- **Separation of Prompts:** Inputs are typically divided into a "system prompt" (instructions, rules, persona) and a "user prompt" (the actual data or query).
- **Prefixes and Suffixes:** When running a dataset through a tool like Promptheus, you can automatically apply prefixes (like a `<User Input>` tag) and suffixes to every single test case so they do not have to be added manually.
- **Security Protocols:** It is crucial to add a security protocol section within the system rules to handle adversarial attacks and unintentional commands. If an LLM encounters a security threat or an unknown command, it should be instructed to output a specific predefined response (e.g., "Summary Request Denied") rather than breaking the application's downstream data pipeline.

Prompt Versioning and Iteration

- **Identifying Issues:** After testing Version 1 against the dataset, you may find that while the persona and security rules work well, the model might generate unnecessary or unwanted content.
- **Creating Version 2 (V2):** To fix output issues, you must isolate and adjust specific blocks of your prompt. For example, if Version 1 asked for reasoning, sentiment, department, and summary, but the reasoning caused formatting issues, Version 2 can be updated to strictly request *only* the sentiment, department, and summary.
- **Updating Rules and Examples:** When versioning, you must also update the "Few-Shot Examples" and "Rules" to match the new objective. V2 rules might specifically dictate the exact sequence of the JSON keys to ensure perfect formatting.

Storing Prompts While prompt engineering platforms will temporarily save your prompt iterations, industry best practices dictate that finalized prompts should be stored in a dedicated database with clear, proper versioning